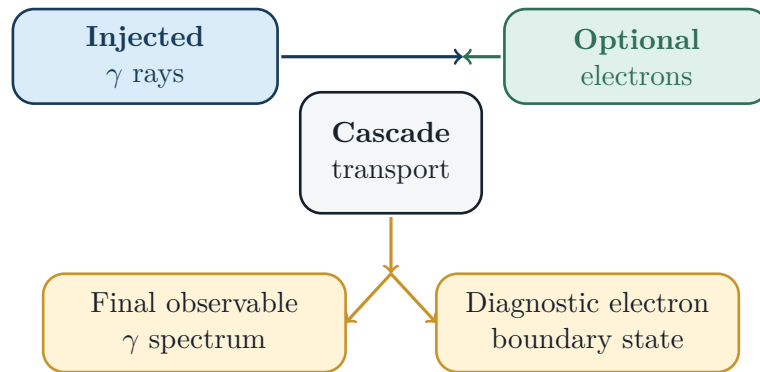


GCascadeV5

Physics-Oriented Code Manual

Gamma rays, electrons, and the structure of the Python implementation



Generated for the repository
/Users/antonio/Desktop/Research/GCascadeV5

April 17, 2026

Contents

1	How to Read This Manual	2
2	The Physical Problem	2
2.1	The Main Equations	2
3	What Users Call	3
4	The Library Tables	4
5	A Visual Map of the Code	4
6	Every File in <code>src/gcascade_v5</code>	5
6.1	<code>__init__.py</code> : the package doorway	5
6.2	<code>core.py</code> : the simple re-export shelf	5
6.3	<code>api.py</code> : the laboratory control panel	5
6.4	<code>state.py</code> : the active observing setup	5
6.5	<code>transport.py</code> : the gamma/electron cascade engine	6
6.6	<code>results.py</code> : the cascade notebook	6
6.7	<code>physics_tables.py</code> : transport-table source checks	7
6.8	<code>physics.py</code> : shared constants, cosmology, and grids	7
6.9	<code>bundle.py</code> : the library archivist	7
6.10	<code>redshift.py</code> : cosmological energy shifting	7
6.11	<code>attenuation.py</code> : absorption without regeneration	8
6.12	<code>config.py</code> : progress messages	8
6.13	<code>tutorial_support.py</code> : small notebook helpers	8
7	Generated Files in the Code Directory	8
7.1	<code>.DS_Store</code>	8
7.2	<code>__pycache__</code> /.	8
8	Repository Folders Outside the Code Directory	8
9	The Flow of <code>CascadePoint</code>	9
10	Testing Commands	9
11	Practical Advice for Future Physics Work	9

1 How to Read This Manual

This guide is written for physicists first. It explains what each file in the code does, but it starts from the physical picture: particles injected at some redshift, interactions with background light, and spectra transported down to redshift zero.

Where a programming word would normally appear, this manual uses a more physical phrase. For example, instead of saying “API”, it says *user-facing functions*. Instead of saying “cache”, it says *recently remembered table slices*. The goal is that the code feels like a set of instruments for evolving spectra, not a maze of software machinery.

Most useful files for physicists

- `api.py`: the functions you call from a notebook.
- `physics.py`: physical constants, cosmology, and the shared grids.
- `transport.py`: the new gamma/electron cascade evolution.
- `state.py`: the active library tables and current EBL/magnetic-field settings.
- `bundle.py`: the runtime library layout and manifest handling.

2 The Physical Problem

GCascadeV5 evolves spectra on a fixed logarithmic energy grid,

$$E_i \in [10^{-1}, 10^{12}] \text{ GeV}, \quad i = 0, \dots, 299.$$

The redshift path is divided into many small steps. At each step the code updates two spectra:

$$\Phi_\gamma(E_i, z), \quad \Phi_e(E_i, z).$$

The previous on-the-spot approximation kept only Φ_γ . Whenever a gamma ray pair-produced an electron/positron pair, the electron energy was assumed to return instantly to photons by inverse Compton scattering at the same redshift step. V5 now keeps the electron spectrum alive:

$$\gamma \longrightarrow e^\pm, \quad e^\pm \longrightarrow e^\pm + \gamma,$$

so delayed cooling, repeated scattering, and magnetic-field losses can be represented more faithfully.

One redshift step, in words

1. Gamma rays survive or pair-produce on the CMB/EBL.
2. Absorbed gamma-ray power is deposited into electrons.
3. Electrons inverse-Compton scatter, producing photons and lower-energy electrons.
4. A magnetic field may remove some electron energy as synchrotron loss.
5. Both spectra are redshifted to the next lower redshift window.

2.1 The Main Equations

A convenient way to read the code is to view one short propagation interval as a set of linear spectral maps.

Photon survival through pair production is treated as

$$\Phi_{\gamma}^{\text{surv}}(E) = \exp[-\Delta s / \lambda_{\gamma\gamma}(E, z)] \Phi_{\gamma}(E),$$

where $\lambda_{\gamma\gamma}$ is the pair-production mean free path and Δs is the propagation length represented by the fine step.

The absorbed photon spectrum is

$$\Delta\Phi_{\gamma}(E) = \Phi_{\gamma}(E) - \Phi_{\gamma}^{\text{surv}}(E).$$

It is redistributed into electrons by the pair-production kernel $K_{\gamma \rightarrow e}$:

$$\Phi_e \leftarrow \Phi_e + K_{\gamma \rightarrow e} \Delta\Phi_{\gamma}.$$

Electrons inverse-Compton scatter with a similar survival factor,

$$\Phi_e^{\text{surv}}(E) = \exp[-\Delta s / \lambda_{\text{IC}}(E, z)] \Phi_e(E).$$

The scattered part,

$$\Delta\Phi_e = \Phi_e - \Phi_e^{\text{surv}},$$

feeds both photons and outgoing electrons:

$$\Phi_{\gamma} \leftarrow \Phi_{\gamma} + K_{e \rightarrow \gamma} \Delta\Phi_e,$$

$$\Phi_e \leftarrow \Phi_e^{\text{surv}} + K_{e \rightarrow e} \Delta\Phi_e.$$

When a magnetic field is active, synchrotron losses compete with ICS. The code uses the same physical scaling as the previous V5 implementation:

$$\left. \frac{dE}{dt} \right|_{\text{syn}} = \frac{1}{e} \frac{4}{3} \sigma_{Tc} \frac{B(z)^2}{2\mu_0} \gamma_e^2, \quad B(z) = B_0(1+z)^{\alpha}.$$

The removed energy is not injected into the gamma-ray spectrum; it is recorded as diagnostic synchrotron energy loss.

3 What Users Call

The most important user-facing functions are:

Function	Physical meaning
<code>RedshiftPoint</code>	Only cosmological redshifting of a point-source spectrum.
<code>AttenuatePoint</code>	Redshifting plus pair-production absorption.
<code>CascadePoint</code>	Full gamma/electron cascade transport for one source.
<code>RedshiftDiffuse</code>	Redshift-only diffuse population.
<code>AttenuateDiffuse</code>	Absorbed diffuse population.
<code>CascadeDiffuse</code>	Full cascade for non-evolving diffuse population.
<code>CascadeEvolving</code>	Full cascade for redshift-dependent injection.

By default, the cascade functions return only the final gamma-ray spectrum, so old notebooks keep working:

$$\phi_{\gamma}(E) = \text{CascadePoint}(\text{gamma_inj}, z).$$

If the full state is desired:

```
return_state=True
```

returns a `CascadeResult` with final photons, final electrons, and diagnostics.

How to think about the electron output

`result.electron` is a diagnostic extragalactic boundary-state spectrum. It is not a direct charged-particle flux at Earth. Galactic magnetic fields, angular spreading, time delays, and local transport are outside this code.

4 The Library Tables

The code reads precomputed physics tables from `LibrariesV5`. The current library format is HDF5, with one manifest:

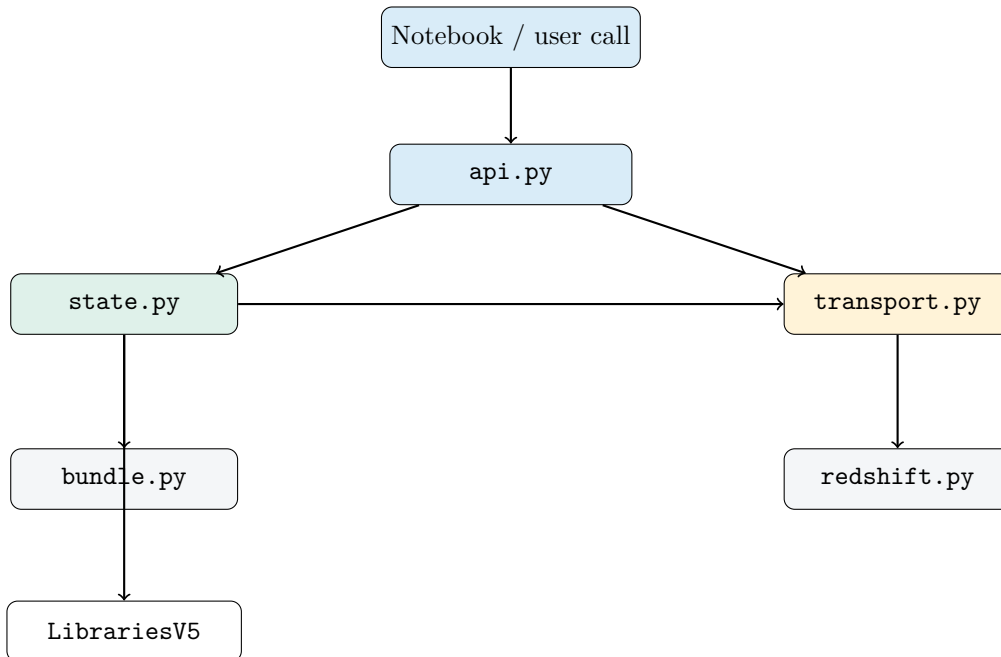
`LibrariesV5/bundle_manifest.json`.

The schema-v2 runtime files include:

- pair-production attenuation tables,
- pair-production spectra,
- ICS interaction rates,
- ICS photon spectra,
- outgoing-electron ICS spectra,
- ICS energy-loss rates for checks and low-energy fallback.

All ICS-related tables are now part of `LibrariesV5` itself. Ordinary cascade runs read one library only: the main HDF5 bundle.

5 A Visual Map of the Code



6 Every File in `src/gcascade_v5`

6.1 `__init__.py`: the package doorway

This is the doorway into the package. When a notebook says

```
import gcascade_v5 as gc,
```

this file decides what appears under the name `gc`. It exposes the main functions and live status values such as the currently selected EBL model.

Physical role: none directly. It is the front door to the instruments.

Important detail: `gc.EBLindex` is not a fixed number copied at import time. It always reflects the currently active EBL model.

6.2 `core.py`: the simple re-export shelf

This file is intentionally tiny. It gathers the main user-facing functions from `api.py`.

Physical role: none directly. It keeps the import surface tidy.

6.3 `api.py`: the laboratory control panel

This is where physicists interact with the code. It checks that spectra have the right shape, chooses the requested calculation, asks `state.py` for the right physics tables, and applies the final point-source or diffuse normalization.

For point sources, the final photon spectrum is scaled by the luminosity distance:

$$\phi_{\gamma}(E_0) = \frac{(1 + z_s)^2}{4\pi d_L^2} \Phi_{\gamma}(E_0, z = 0).$$

Diffuse calculations use the usual $1/(4\pi)$ angular factor after integrating the source distribution.

Main functions:

- `RedshiftPoint`, `RedshiftDiffuse`, `RedshiftEvolving`
- `AttenuatePoint`, `AttenuateDiffuse`, `AttenuateEvolving`
- `CascadePoint`, `CascadeDiffuse`, `CascadeEvolving`
- `changeEBLModel`, `changeMagneticField`
- `get_bundle_info`, `reset_factory_settings`

How to read it: search for the function you call from a notebook. The body of that function shows the calculation sequence in plain order: validate input, load tables, propagate, normalize, return.

6.4 `state.py`: the active observing setup

This file remembers which EBL model is active, which HDF5 library is being used, which redshift tables are loaded, and what magnetic-field parameters are currently selected.

An analogy: if `api.py` is the control panel, `state.py` is the current experimental setup. It knows which target, filters, and calibration tables are installed.

Important physical quantities held here:

- pair-production attenuation factors,

- ICS attenuation factors,
- redshift interpolation tables,
- pair-production spectra,
- ICS photon and electron spectra,
- B_0 and α in $B(z) = B_0(1+z)^\alpha$.

Because the HDF5 kernels are large, `state.py` only keeps a few recently used redshift slices ready at a time. This is a practical memory-saving detail, not a change in the physics.

6.5 `transport.py`: the gamma/electron cascade engine

This is the physical heart of the new V5 cascade. It advances

$$\begin{pmatrix} \Phi_\gamma \\ \Phi_e \end{pmatrix}$$

from high redshift to low redshift.

Central operation:

$$\begin{aligned} \Phi_\gamma &\rightarrow \Phi_\gamma^{\text{surv}} + K_{e\rightarrow\gamma}\Delta\Phi_e, \\ \Phi_e &\rightarrow \Phi_e^{\text{surv}} + K_{\gamma\rightarrow e}\Delta\Phi_\gamma + K_{e\rightarrow e}\Delta\Phi_e. \end{aligned}$$

Main functions:

- `transport_step`: one fixed-redshift transport step.
- `propagate_point_transport`: point-source cascade.
- `propagate_diffuse_transport`: diffuse cascade.
- `propagate_evolving_transport`: redshift-evolving source cascade.
- `synchrotron_loss_rates`: computes synchrotron loss for $B(z)$.
- `spectrum_energy`: integrates a spectrum over the energy grid.

Low-energy behavior: when discrete ICS scattering becomes too small relative to the logarithmic energy bin width, the code uses a conservative continuous-loss style fallback so electrons do not get numerically stuck in a single bin.

6.6 `results.py`: the cascade notebook

This file defines `CascadeResult`, the object returned when `return_state=True`. It contains:

- `gamma`: the final gamma-ray spectrum,
- `electron`: the diagnostic electron spectrum,
- `diagnostics`: energy bookkeeping,
- `metadata`: run labels and normalization information.

Physical role: it keeps all final spectra and bookkeeping in one place so the gamma observable and electron diagnostic do not get confused.

One diagnostic deserves special mention: `redshift_energy_lost`. This is not a numerical error. It is the energy removed by cosmological expansion while the particles slide from one redshift window to the next.

6.7 `physics_tables.py`: transport-table source checks

This file checks that a `LibrariesV5` bundle contains the transport tables needed by the gamma+electron cascade and loads them when the bundle is used as a source during library rebuilding.

Physical tables checked here:

- `ics_imfp`: ICS interaction rates,
- `ics_gamma_packed`: ICS photon spectra,
- `ics_electron_packed`: outgoing-electron spectra,
- `pp_packed`: pair-production spectra,
- `dEdt_ics`: ICS loss-rate tables.

6.8 `physics.py`: shared constants, cosmology, and grids

This file collects the physical constants and common grids used throughout the code:

- the electron rest mass,
- the Thomson cross section,
- the present CMB temperature,
- the logarithmic energy grid,
- the pretabulated redshift windows,
- the Hubble expansion law and luminosity distance.

Why it matters physically: it gives one single place where the basic background assumptions of the calculation are defined.

6.9 `bundle.py`: the library archivist

This file describes the HDF5 library format and reads the manifest that tells the code where each runtime table lives.

Physical content written into runtime files:

- $\lambda_{\gamma\gamma}^{-1}$ and gamma survival factors,
- $K_{\gamma\rightarrow e}$ pair-production spectra,
- λ_{IC}^{-1} and electron survival factors,
- $K_{e\rightarrow\gamma}$ ICS photon spectra,
- $K_{e\rightarrow e}$ outgoing-electron spectra,
- dE/dt ICS loss-rate tables.

The file also records provenance in `bundle_manifest.json`, so one can see where the runtime tables came from.

6.10 `redshift.py`: cosmological energy shifting

This file applies the mapping between neighboring redshift windows. In physical terms,

$$E_{\text{emitted}} = E_{\text{observed}} \frac{1 + z_{\text{high}}}{1 + z_{\text{low}}}.$$

The spectrum is interpolated in logarithmic space, which is appropriate for the wide logarithmic energy grid.

Used by: redshift-only calculations, attenuation calculations, and the new gamma/electron transport.

6.11 `attenuation.py`: absorption without regeneration

This file evolves photons with pair-production absorption but without secondary cascade regeneration. The basic operation is:

$$\Phi_{\gamma}(E) \leftarrow \exp[-\tau_{\gamma\gamma}(E)]\Phi_{\gamma}(E).$$

It is useful when one wants to separate:

$$\text{redshift only} \rightarrow \text{redshift} + \text{absorption} \rightarrow \text{full cascade}.$$

6.12 `config.py`: progress messages

This file controls:

- whether progress messages are shown,
- simple progress bars.

Physical role: none. It affects convenience and speed only.

6.13 `tutorial_support.py`: small notebook helpers

This file contains small functions used by tutorials to check whether a bundle exists and where its manifest should be. It has no cascade physics.

7 Generated Files in the Code Directory

7.1 `.DS_Store`

A macOS Finder metadata file. It has no physical or numerical meaning.

7.2 `__pycache__`

Python creates these files automatically after imports. They can be deleted safely; the computer will regenerate them when needed.

8 Repository Folders Outside the Code Directory

Folder	Meaning
<code>LibrariesV5</code>	Main HDF5 physics library used by normal runs.
<code>tests</code>	Checks for physics-table loading, transport behavior, and energy conservation.
<code>benchmarks</code>	Runtime timing helpers.
<code>tutorial.ipynb</code>	Interactive tutorial for users.
<code>safe_snapshots</code>	Preserved baseline copy of the previous V5 Python code.

9 The Flow of CascadePoint

1. The user supplies a gamma spectrum, optional electron spectrum, and source redshift.
2. `api.py` checks shapes and redshift range.
3. `state.py` opens the active EBL tables in `LibrariesV5`.
4. `transport.py` evolves Φ_γ and Φ_e down the redshift grid.
5. `redshift.py` remaps both spectra between redshift windows.
6. `api.py` applies the luminosity-distance factor.
7. The user receives either ϕ_γ or the full `CascadeResult`.

10 Testing Commands

Default checks:

```
GCASCADE_PROGRESS=0 PYTHONPATH=src python3 -m pytest -q.
```

Focused electron-transport checks:

```
PYTHONPATH=src python3 -m pytest -q tests/test_transport.py.
```

11 Practical Advice for Future Physics Work

Where to modify the code

- Change cascade physics in `transport.py`.
- Add new HDF5 table types through `bundle.py` and `state.py`.
- Add new transport-table validation rules in `physics_tables.py`.
- Add user-facing controls in `api.py`.
- Keep the energy bookkeeping tests updated whenever transport physics changes.

Most important conceptual distinction

Old V5: photons only, electrons cool immediately.

New V5: photons and electrons are both evolved.

This is the central structural change. Most code changes follow from this one physics decision.